

FIA/P GRADUAÇÃO

DOMAIN DRIVEN DESIGN

Prof. Me. Thiago T. I. Yamamoto

#15 – WEB SERVICES RESTFUL - API



TRAJETÓRIA



#15 - AGENDA



- **Integrações de Sistemas**
 - Arquitetura de aplicações
- **Web Services Restful**
 - URI, Métodos e Status Code do HTTP
- **Web Services Restful com Java**
 - JAS-RS, principais anotações
 - Criação do projeto e configuração
 - Implementação do CRUD com Json
- **Consumindo um Web Services Restful**
 - Criando e configurando o projeto
 - Consumindo o CRUD

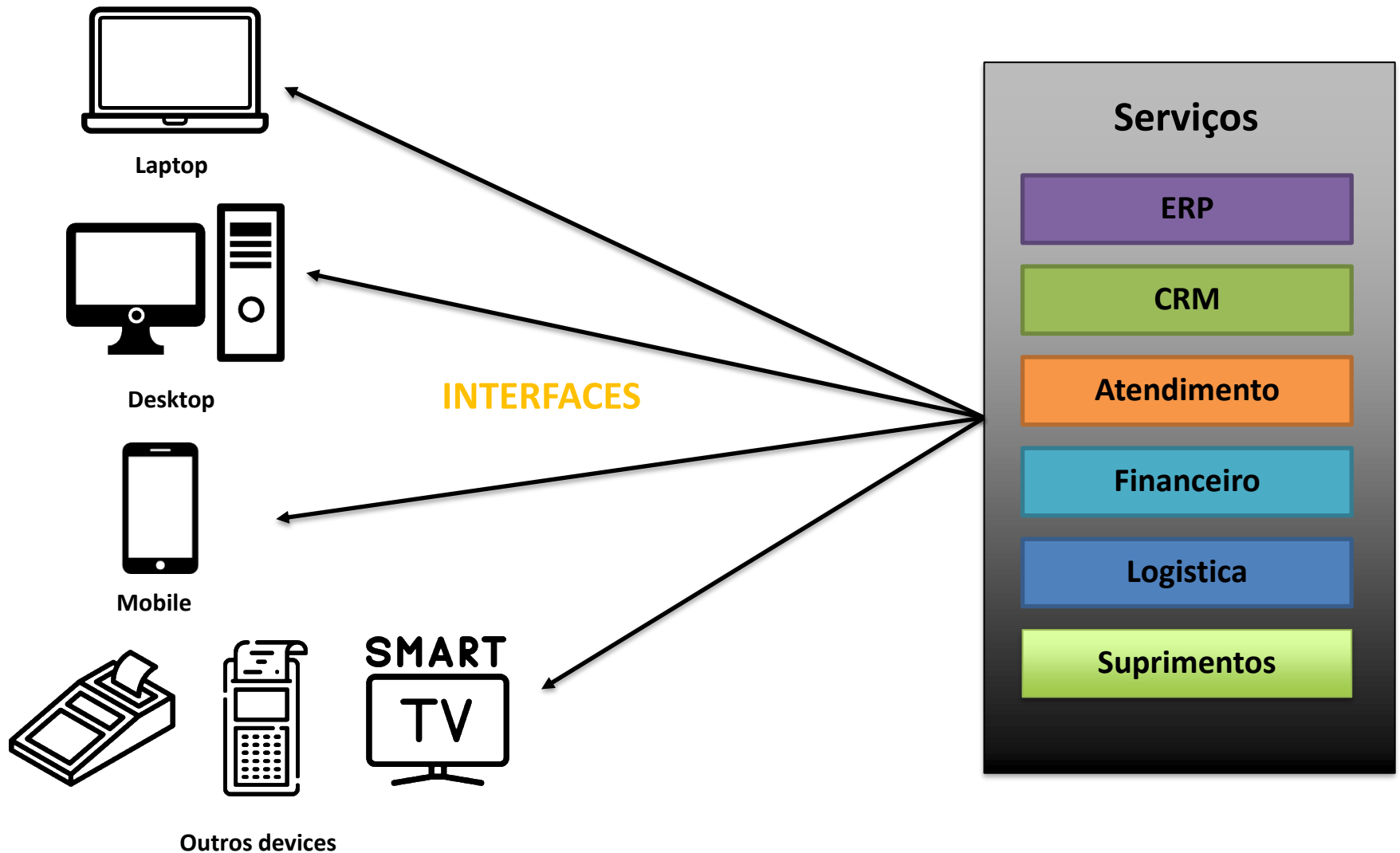
INTEGRAÇÃO DE APLICAÇÕES

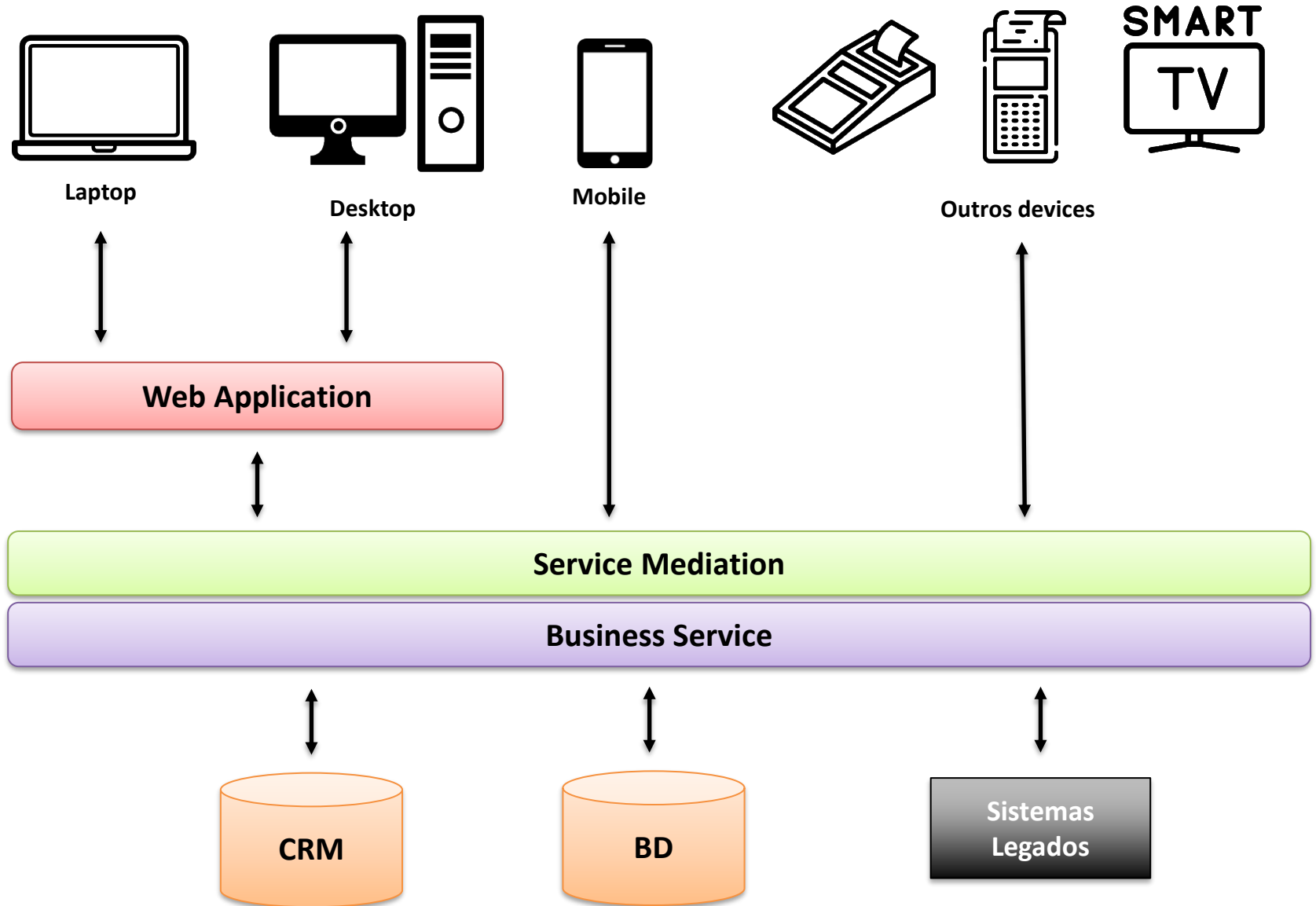
Departamentos de uma empresa

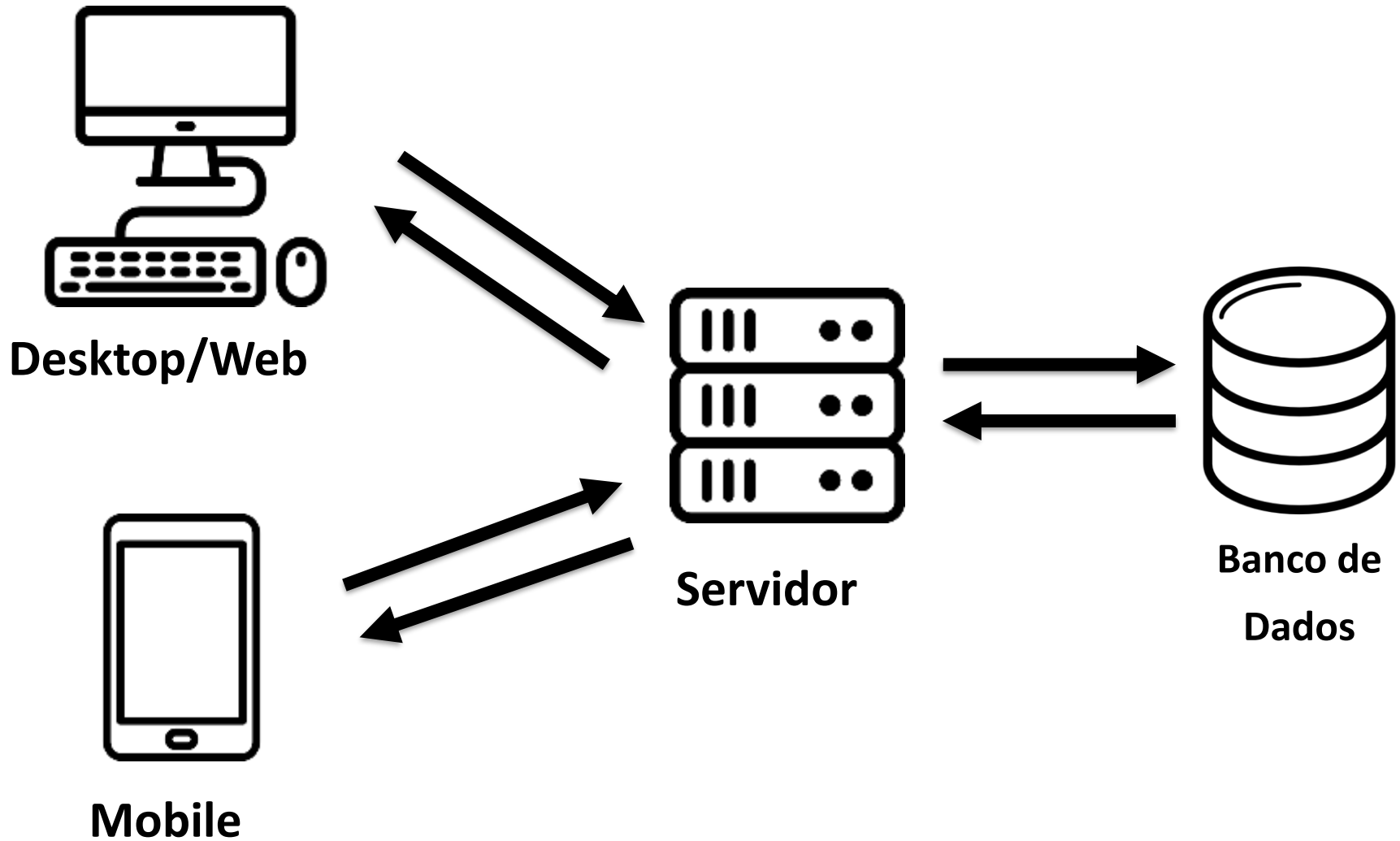


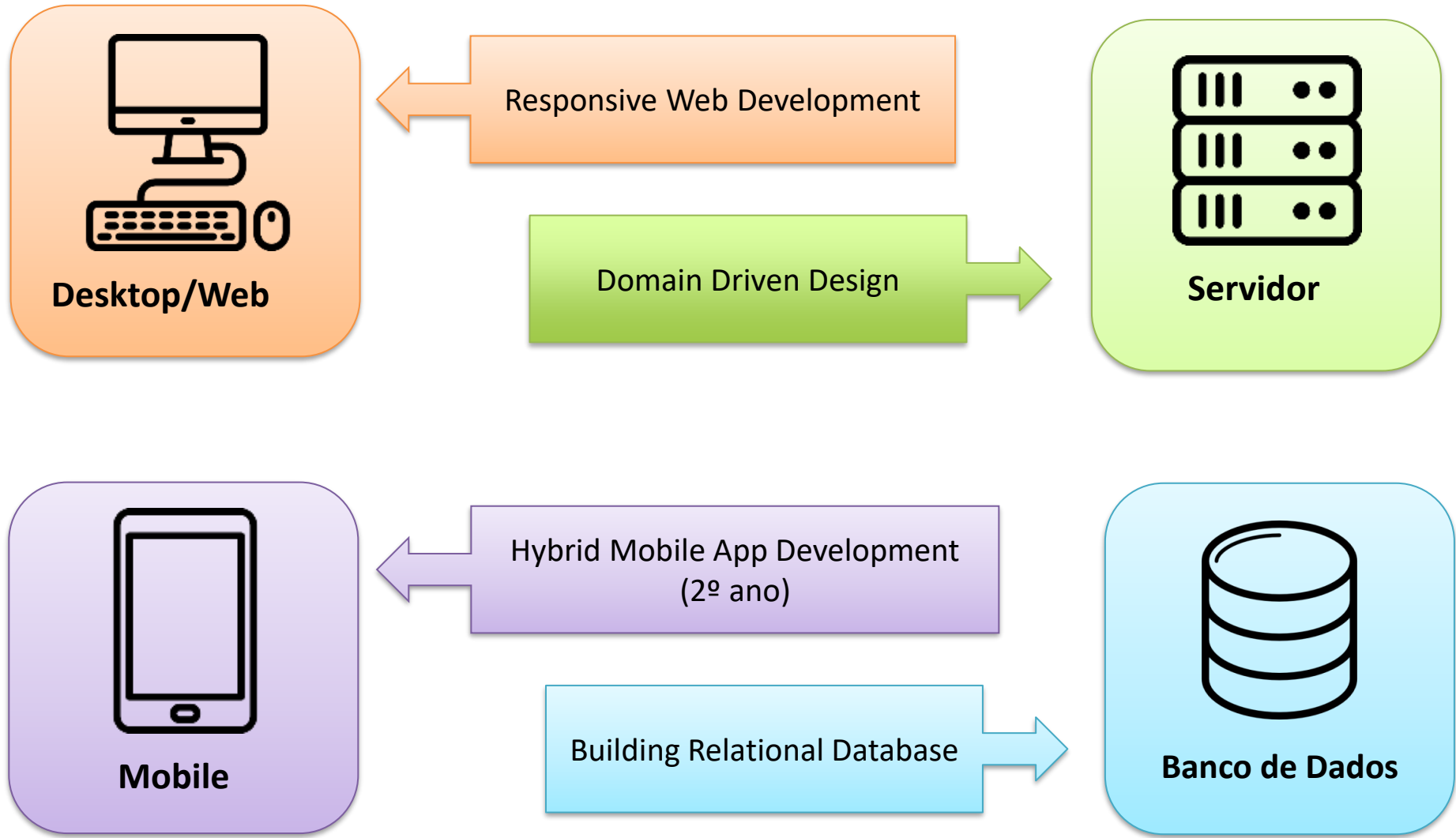
Integração entre empresas

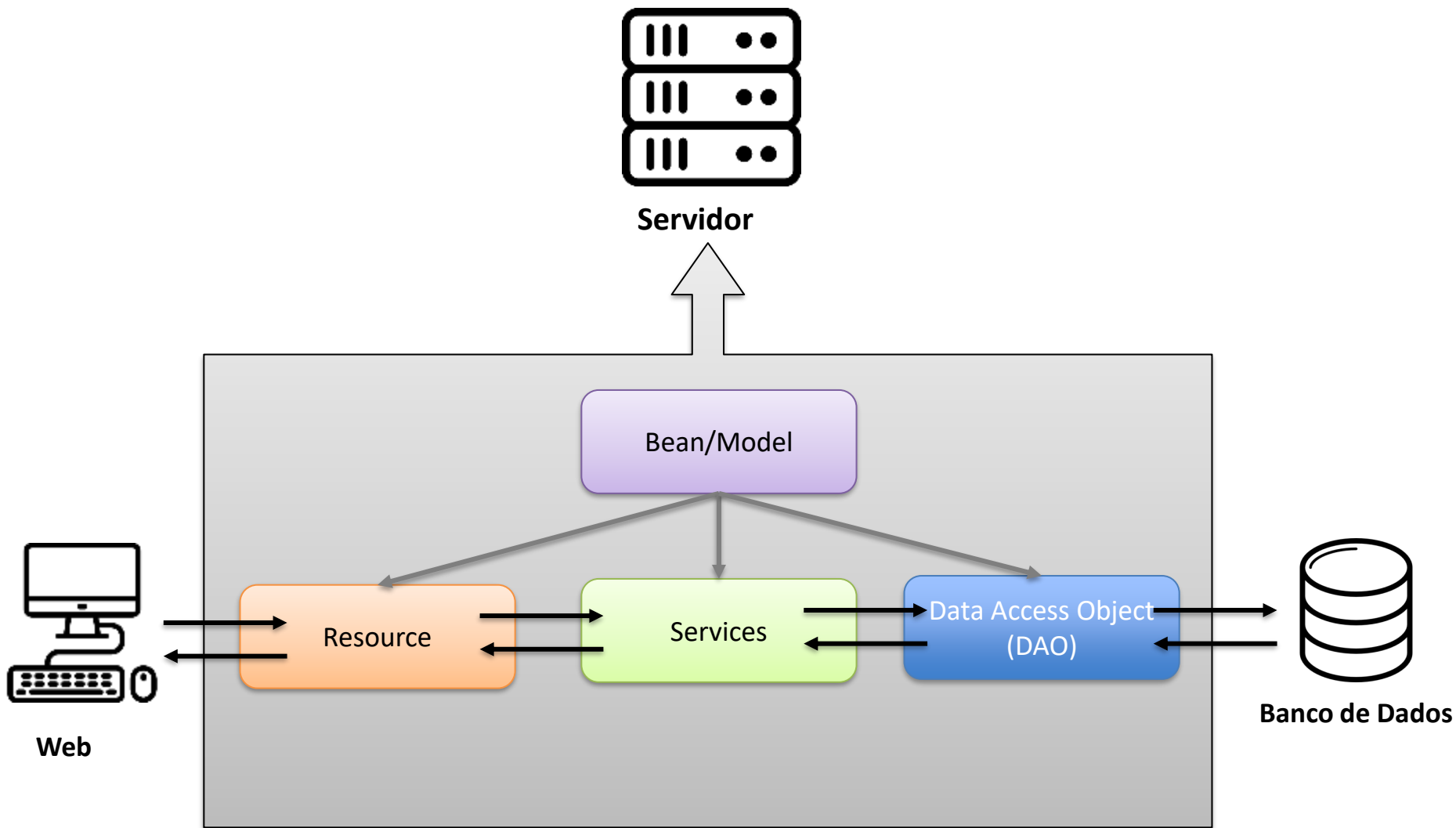








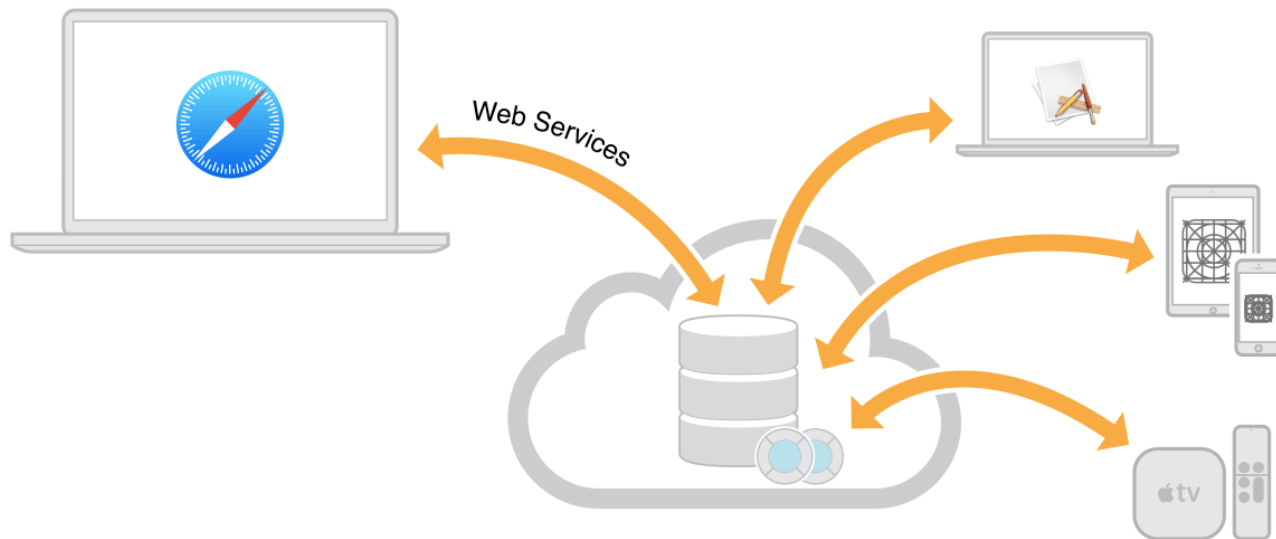






WEB SERVICES

- **Integração e comunicação** entre sistemas diferentes;
- **Independente de plataforma** (Java, .NET, PHP, Ruby, etc..)
- Permite o envio e recebimento de dados em formatos **XML, Json, CSV**, etc..
- Os dois tipos mais comuns de web services: **SOAP** e **REST**;



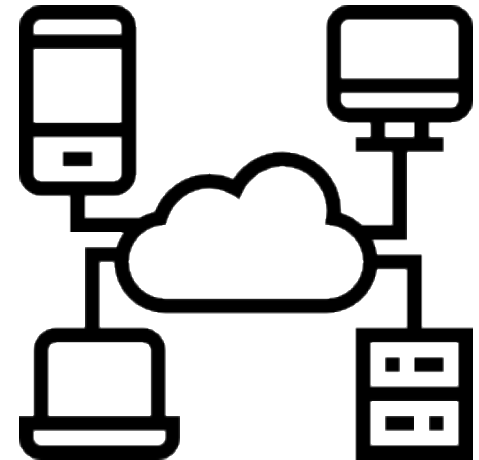
RESTFul (REpresentational State Transfer)

- Simples, leve, fácil de desenvolver e evoluir;
- Tudo é um recurso (**Resource**);
- Cada recurso possui um identificador (**URI**);
- Recursos podem utilizar vários formatos: html, xml, **Json**;
- Utiliza o Protocolo **HTTP**;
- Os métodos HTTP: **GET, POST, PUT, DELETE** são utilizados na arquitetura REST.



O protocolo **HTTP** possui vários métodos, os principais:

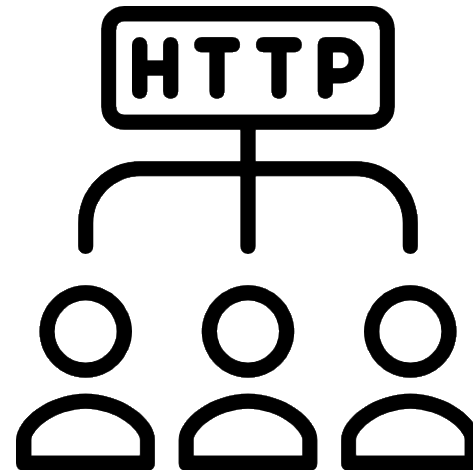
- **GET:** recupera informações de um recurso;
- **POST:** cria um novo recurso;
- **PUT:** atualiza um recurso;
- **DELETE:** remove um recurso;



- Quando realizamos uma requisição, é preciso determinar o **endereço** do **recurso** que vamos acessar:

VERBO	URI	AÇÃO
POST	/pedido/	Criar
GET	/pedido/1	Visualizar
PUT	/pedido/1	Alterar
DELETE	/pedido/1	Apagar

- Os códigos de **status das respostas HTTP** indicam se uma requisição HTTP foi corretamente concluída. As respostas são agrupadas em cinco classes: **respostas de informação, respostas de sucesso, redirecionamentos, erros do cliente e erros do servidor**;
- **1xx** – Informativa;
- **2xx** – Sucesso;
- **3xx** – Redirecionamentos;
- **4xx** – Erros do cliente;
- **5xx** – Erros do servidor;



- Vamos trabalhar com alguns códigos na implementação do **Web Service**:

CODE	DESCRIÇÃO
200	Ok
201	Created (Criado)
204	No Content (Sem conteúdo)
500	Internal Server Error
404	Not Found
405	Method not Allowed



QUARKUS

O Quarkus é um framework Java moderno, desenvolvido para criar aplicações de alta performance, com baixo consumo de memória e tempo de inicialização extremamente rápido. Ele foi projetado especialmente para ambientes cloud-native, containers e arquiteturas baseadas em microserviços.

Vantagens:

- Alta performance
- Baixo consumo de memória
- Tempo de inicialização extremamente rápido
- Execução eficiente em containers e ambientes serverless

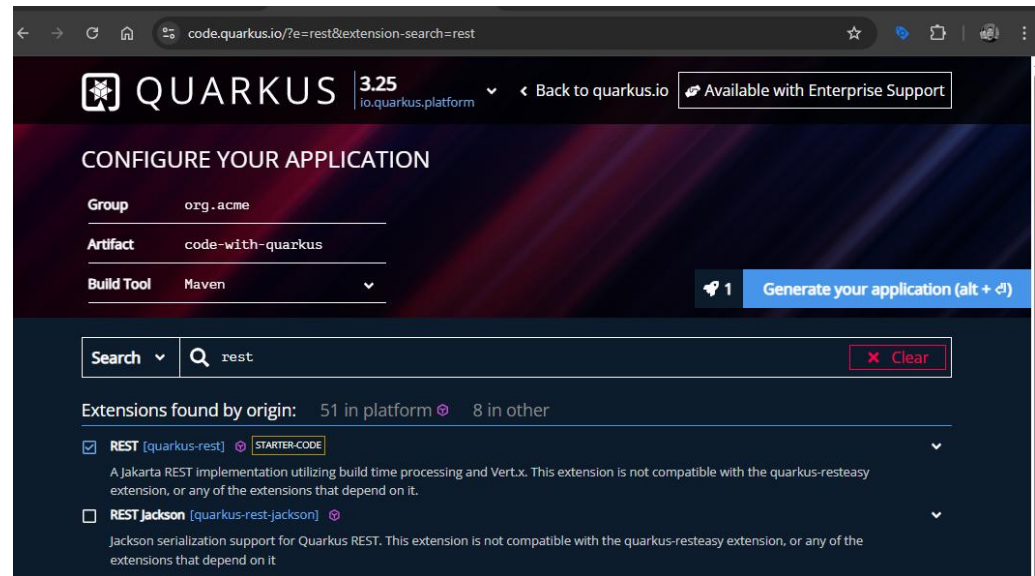


<https://code.quarkus.io/>



CRIANDO O PROJETO

- Acesse: <https://code.quarkus.io>
- Preencha:
 - **Group Id:** *br.com.fiap*
 - **Artifact:** *projeto-api*
- Adicione extensões:
 - REST
 - REST Jackson
 - JDBC Oracle
- Clique em "Generate your application"
- Extraia e abra o projeto no IntelliJ



- Adicione configuração de execução
 - Vá em Run > Edit Configurations...
 - Clique em + > selecione Maven
 - Em “Name”: *Run QuarkusEm (ou outro que desejar)*
 - “Working directory”: selecione o diretório do projeto
 - Em “Command”: **quarkus:dev**
 - Clique em OK
- Execute a aplicação
 - Clique no botão verde  ao lado da nova configuração *Run Quarkus*
 - O projeto será iniciado no modo dev:

 Acesse: <http://localhost:8080>



WEB SERVICE RESTFUL COM JAVA

JAX-RS

- Especificação Java para suporte a REST (JSR 331);
- **JAX-RS**: Java API for RESTFul Web Services;
- **RESTEasy Reactive**: implementação da especificação.
- O Quarkus adota o padrão **JAX-RS** com uma **implementação moderna, reativa e otimizada**.

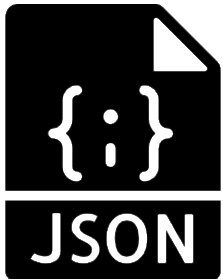
Principais anotações:

Anotação	Descrição
@Path	Define o caminho para o recurso (URI).
@POST	Responde por requisições POST.
@GET	Responde por requisições GET.
@PUT	Responde por requisições PUT.
@DELETE	Responde por requisições DELETE.
@Produces	Define o tipo de informação que o recurso retorna.
@Consumes	Define o tipo de informação que o recurso recebe.
@PathParam	Injeta um parâmetro da URL no parâmetro do método.

- Ao invés de trabalhar com o envio de texto puro, é melhor trabalhar com **Json – JavaScript Object Notation**;
- Formato simples e leve para transferência de dados;
- Uma alternativa para o XML;

Exemplo:

```
{  
  "show": "Oasis",  
  "preco": 150,  
  "local": "São Paulo"  
}
```



Exemplo de um array em JSON:

```
{
  "shows": [
    {
      "show": "Oasis",
      "preco": 150,
      "local": "São Paulo"
    },
    {
      "show": "Link Park",
      "preco": 250,
      "local": "Rio de Janeiro"
    },
    {
      "show": "Jorge e Mateus",
      "preco": 200,
      "local": "São Paulo"
    }
  ]
}
```

{ JSON }

Os colchetes [] limitam o array.



ACESSO AOS DADOS

- Crie a classe de modelo, que representa uma tabela no banco de dados.

```
package br.com.fiap.model;

public class Aluno {
    private int id;
    private String nome;
    private String email;

    public Aluno() {}

    public Aluno(int id, String nome, String email) {
        this.id = id;
        this.nome = nome;
        this.email = email;
    }

    //getters e setters..
}
```

- Execute os comandos para gerar a tabela e sequence no banco de dados:

```
CREATE TABLE aluno (  
  id NUMBER PRIMARY KEY,  
  nome VARCHAR2(100) NOT NULL,  
  email VARCHAR2(100) NOT NULL  
);  
  
CREATE SEQUENCE aluno_seq START WITH 1 INCREMENT BY 1;
```

CONFIGURAÇÃO DO PROJETO PARA ACESSAR O BANCO FIAP

- No arquivo **application.properties**, configure os dados do banco:

```
quarkus.datasource.db-kind=oracle  
quarkus.datasource.username=usuario  
quarkus.datasource.password=senha  
quarkus.datasource.jdbc.url=jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl  
quarkus.datasource.jdbc.driver=oracle.jdbc.OracleDriver  
  
quarkus.http.port=8080
```

- Agora, basta implementar o **DAO** com os comandos básicos (**CRUD**):

```
package br.com.fiap.dao;

import br.com.fiap.model.Aluno;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import javax.sql.DataSource;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

@ApplicationScoped
public class AlunoDao {

    @Inject
    DataSource dataSource;

}
```


- **@ApplicationScoped:** define que o DAO é instanciado uma vez para toda a aplicação;
- **@Inject DataSource:** injeta a conexão configurada com o banco Oracle.

Com essas anotações, o desenvolvedor não precisa se preocupar em instanciar os objetos.

- Busque por todos os alunos no banco de dados e retorne em uma lista.

```
public List<Aluno> listar() throws SQLException {  
    List<Aluno> lista = new ArrayList<>();  
    try (Connection conn = dataSource.getConnection();  
        Statement stmt = conn.createStatement();  
        ResultSet rs = stmt.executeQuery("SELECT id, nome, email FROM aluno")) {  
  
        while (rs.next()) {  
            lista.add(new Aluno(rs.getInt("id"), rs.getString("nome"), rs.getString("email")));  
        }  
    }  
    return lista;  
}
```

- Busque por um aluno pelo Id, caso não encontre, lance uma exception.

```
public Aluno buscar(int id) throws SQLException {  
    try (Connection conn = dataSource.getConnection();  
        PreparedStatement stmt = conn.prepareStatement  
            ("SELECT id, nome, email FROM aluno WHERE id = ?")) {  
        stmt.setInt(1, id);  
        ResultSet rs = stmt.executeQuery();  
        if (!rs.next()) {  
            throw new EntityNotFoundException();  
        }  
        return new Aluno(rs.getInt("id"), rs.getString("nome"), rs.getString("email"));  
    }  
}
```

- Cadastre um aluno, atualizando o objeto aluno com o id gerado.

```
public void inserir(Aluno aluno) throws SQLException {  
    try (Connection conn = dataSource.getConnection();  
        PreparedStatement stmt = conn.prepareStatement(  
            "INSERT INTO aluno (id, nome, email) VALUES (aluno_seq.NEXTVAL, ?, ?)",  
            new String[] { "id" }))) {  
  
        stmt.setString(1, aluno.getNome());  
        stmt.setString(2, aluno.getEmail());  
        stmt.executeUpdate();  
  
        ResultSet rs = stmt.getGeneratedKeys();  
        if (rs.next()) {  
            int id = rs.getInt(1);  
            aluno.setId(id);  
        }  
    }  
}
```

- Atualize um aluno.

```
public void atualizar(Aluno aluno) throws SQLException {  
    try (Connection conn = dataSource.getConnection();  
        PreparedStatement stmt = conn.prepareStatement  
            ("UPDATE aluno SET nome = ?, email = ? WHERE id = ?")) {  
        stmt.setString(1, aluno.getNome());  
        stmt.setString(2, aluno.getEmail());  
        stmt.setInt(3, aluno.getId());  
        stmt.executeUpdate();  
    }  
}
```

- Remova um aluno.

```
public void deletar(int id) throws SQLException {  
    try (Connection conn = dataSource.getConnection();  
        PreparedStatement stmt = conn.prepareStatement  
            ("DELETE FROM aluno WHERE id = ?")) {  
        stmt.setInt(1, id);  
        stmt.executeUpdate();  
    }  
}
```



API

IMPLEMENTAÇÃO DOS ENDPOINTS

- Crie a classe para a implementação dos Endpoints!

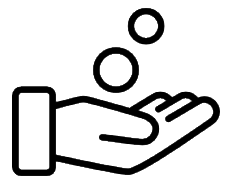
```
package br.com.fiap.resource;

import br.com.fiap.dao.AlunoDao;
import br.com.fiap.model.Aluno;
import jakarta.inject.Inject;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.Context;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import jakarta.ws.rs.core.UriInfo;
import java.net.URI;
import java.sql.SQLException;
import java.util.List;

@Path("/alunos")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class AlunoResource {

    @Inject
    AlunoDao dao;

}
```



- **@Path("/alunos"):** Define a URL base para acessar os métodos dessa classe.
Exemplo: GET <http://localhost:8080/alunos>
- **@Produces(MediaType.APPLICATION_JSON):** Informa que a resposta da API será enviada em formato JSON.
- **@Consumes(MediaType.APPLICATION_JSON):** Indica que os dados recebidos pela API (ex: via POST) devem estar em JSON.
- **@Inject:** é uma anotação do CDI (Context and Dependency Injection). Permite que o Quarkus injete automaticamente uma instância da classe `AlunoDao`, que acessa o banco de dados. **Assim, você não precisa criar a instância manualmente com new.**

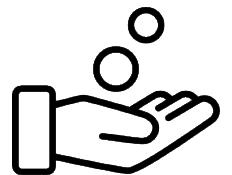


GET - BUSCAS

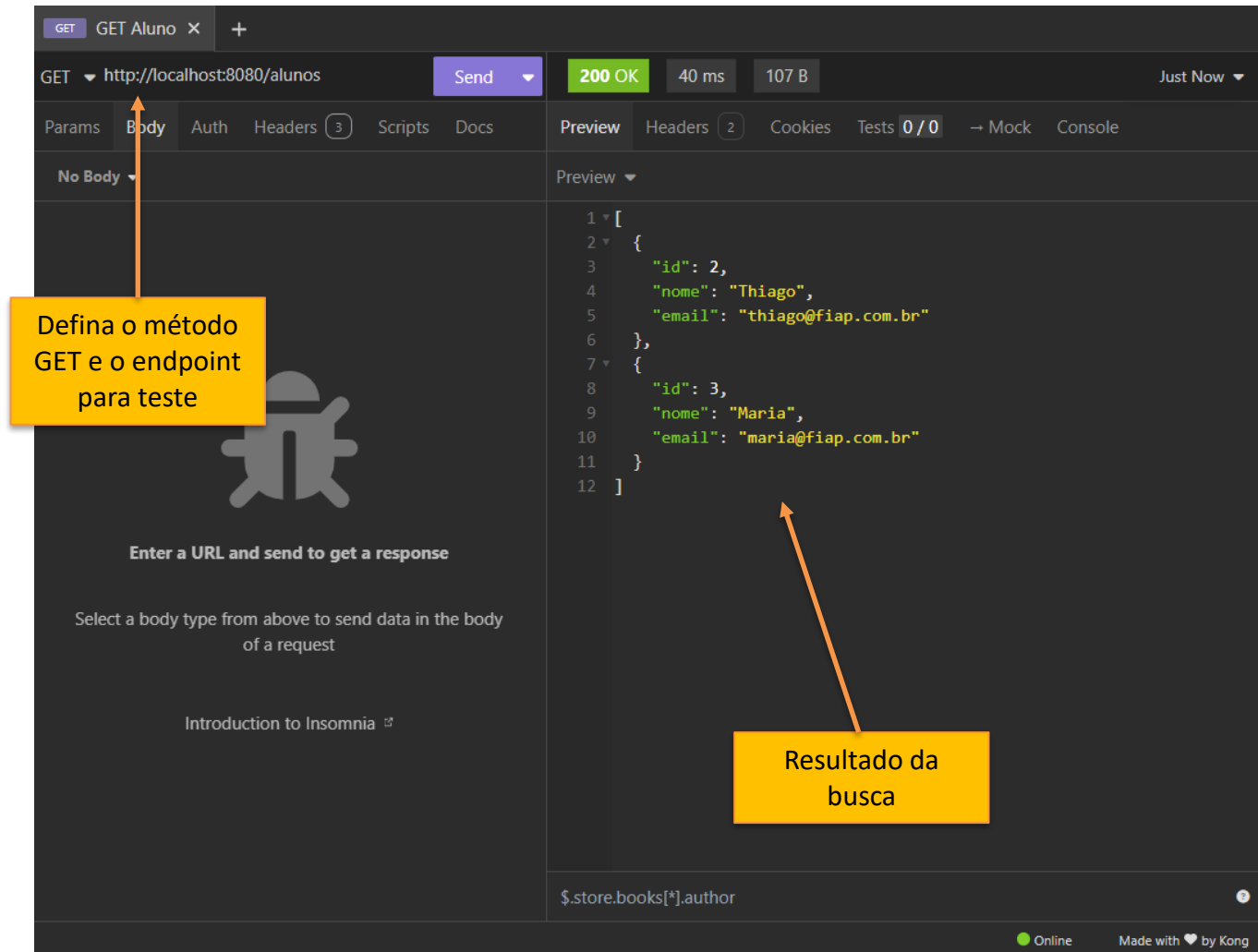
- Desenvolva a **listagem** e a **buscar por id** do aluno:

```
@GET
public List<Aluno> listar() throws SQLException {
    return dao.listar();
}
```

```
@GET
@Path("/{id}")
public Response buscar(@PathParam("id") int id) throws SQLException {
    Aluno aluno = dao.buscar(id);
    return Response.ok(aluno).build();
}
```



- Utilize uma ferramenta como Insomnia ou Postman.



- Adicione o id do aluno no final da URL para a pesquisa:



- Atualmente, quando o ID do aluno não é encontrado no banco, a API lança uma exceção. Esse comportamento será tratado adequadamente em breve.



POST - CADASTRO

- Crie um método para **cadastrar** um Aluno.
- Ao cadastrar o método retorna o status **201 Created**, junto do **recurso criado** e a **URL para acessar o novo recurso**.

```
@POST
public Response adicionar(Aluno aluno, @Context UriInfo uriInfo) throws SQLException {
    dao.inserir(aluno);

    URI uri = uriInfo.getAbsolutePathBuilder()
        .path(String.valueOf(aluno.getId()))
        .build();

    return Response.created(uri)
        .entity(aluno)
        .build();
}
```

The screenshot shows a REST client interface with a dark theme. At the top, a tab is labeled 'POST Aluno'. The main area is divided into two panels. The left panel shows the request details: the method is 'POST' and the URL is 'http://localhost:8080/alunos'. The 'Body' tab is selected, showing a JSON payload:

```
{  "nome" : "Thiago",  "email" : "thiago@fiap.com.br"}
```

. The right panel shows the response details: the status is '201 Created', the time is '1.07 s', and the size is '53 B'. The 'Preview' tab is selected, showing the response JSON:

```
{  "id": 4,  "nome": "Thiago",  "email": "thiago@fiap.com.br"}
```

. Two orange arrows point from the JSON in the request body to the JSON in the response preview.

Defina o método POST e a URL.
Adicione também no corpo da
requisição o JSON com os dados do
Aluno.

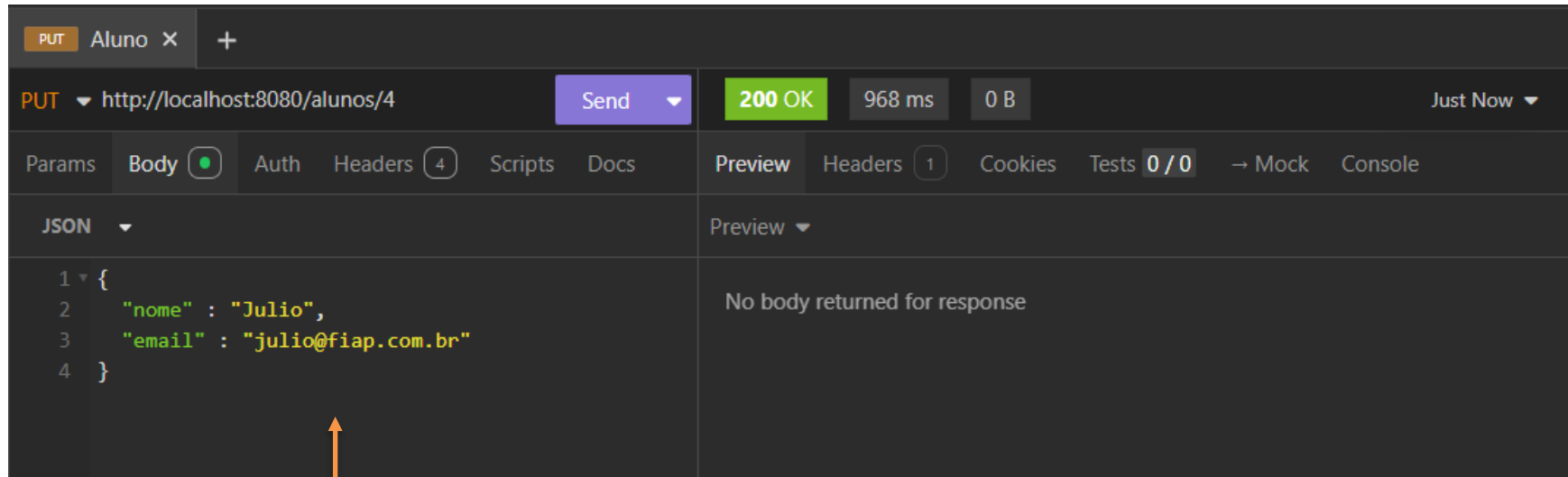
Resultado do Cadastro com status
code 201, o recurso criado e em
Headers a URL para acessar o
recurso.



PUT - ATUALIZAÇÃO

- O método para atualização recebe um objeto **Aluno** e o **id** do aluno que será atualizado.

```
@PUT
@Path("/{id}")
public Response atualizar(@PathParam("id") int id, Aluno aluno) throws SQLException {
    aluno.setId(id);
    dao.atualizar(aluno);
    return Response.ok().build();
}
```



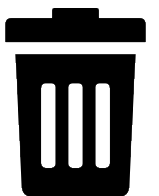
Defina o método PUT e a URL com o Id do aluno. Adicione também no corpo da requisição o JSON com os dados do Aluno.

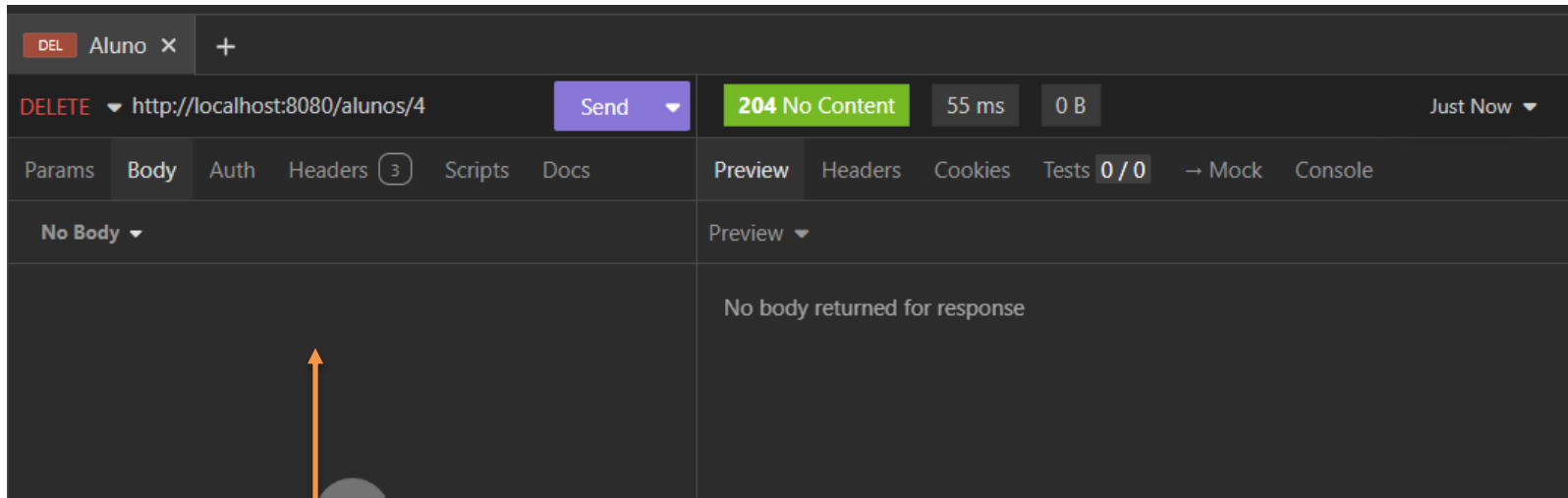


DELETE - REMOÇÃO

- O método de remoção **recebe** um **código** e retorna o código **HTTP 204** (no content));

```
@DELETE
@Path("/{id}")
public Response deletar(@PathParam("id") int id) throws SQLException {
    dao.deletar(id);
    return Response.noContent().build();
}
```





Defina o método DELETE e a URL com o Id do aluno.

Copyright © 2020 - 2025
Prof. Thiago T. I. Yamamoto

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).

*“O que a pessoa pensa a respeito de si mesmo indica o seu destino”.
(Henry David Thoreau, adaptado)*